Universidad de San Carlos de Guatemala

Facultad de Ingeniería

Escuela de Ciencias y Sistemas

Lenguajes formales de Programación

Manual Técnico

Nombre: Carlos Josué Calderón Cabrera

Carnet: 201314753

Fecha: Guatemala, 03 de marzo de 2026

Contenido

Objetivo ……. 3

Metodos y Funcionamientos …….. 4

Objetivo

La presente practica se realizó con el objetivo de hacer uso de los contenidos aprendidos del laboratorio de Lenguajes Formales de Programación, con la finalidad de evaluar los conocimientos aprendidos en la asignatura e implementando la lógica y el uso de algoritmos para resolver este ejercicio.

Metodos y Funcionamiento

Durante la Presente practica se hizo uso de diversos métodos y clases para que el sistema pueda manejar de manera ordenada las solicitudes que el usuario requiera entre la cuales encontramos las siguientes clases:

```cpp
#ifndef NOTA_H
#define NOTA_H


#include <iostream>
#include <string>

class Nota
{
private:
    int carnet;
    int codigo_curso;
    float nota;
    std::string ciclo;
    int anio;
public:
    Nota(int carnet, int codigo_curso, float nota, std::string ciclo, int anio)
    {
        this->carnet = carnet;
        this->codigo_curso = codigo_curso;
        this->nota = nota;
        this->ciclo = ciclo;
        this->anio = anio;
    }
    int getCarnet() const
    {
        return carnet;
    }

    int getCodigoCurso() const
    {
        return codigo_curso;
    }

    float getNota() const
```

Clase nota: esta clase contiene los atributos y metodos de las notas que se ingresaran al sistema

```cpp
#ifndef ESTUDIANTE_H
#define ESTUDIANTE_H


#include <iostream>
#include <string>


class Estudiante
{
private:
    int carnet;
    std::string nombre;
    std::string apellido;
    std::string carrera;
    int semestre;

public:
    Estudiante(int carnet, std::string nombre, std::string apellido, std::string carrera, int semestre)
    {
        this->carnet = carnet;
        this->nombre = nombre;
        this->apellido = apellido;
        this->carrera = carrera;
        this->semestre = semestre;
    }
    int getCarnet() const
    {
        return carnet;
    }

    std::string getNombre() const
    {
        return nombre;
    }

    std::string getApellido() const
```

Clase estudiante: en esta clase contiene los atributos y métodos de los estudiantes que ingresaran al sistema

```cpp
#ifndef CURSO_H
#define CURSO_H


#include <iostream>
#include <string>

class Curso
{
private:
    int codigo;
    std::string nombre;
    int creditos;
    int semestre;
    std::string carrera;
public:
    Curso(int codigo, std::string nombre, int creditos, int semestre, std::string carrera)
    {
        this->codigo = codigo;
        this->nombre = nombre;
        this->creditos = creditos;
        this->semestre = semestre;
        this->carrera = carrera;
    }
    int getCodigo() const
    {
        return codigo;
    }

    std::string getNombre() const
    {
        return nombre;
    }

    int getCreditos() const
    {
        return creditos;
```

La clase curso contiene los atributos y métodos implementados para la asignación de datos al sistema

```
vector<string> split(string linea, char delimiter)
{
    vector<string> compuesto;
    string actual = "";
    for (char c : linea)
    {
        if (c == delimiter)
        {
            compuesto.push_back(actual);
            actual = "";
        }
        else
        {
            actual += c;
        }
    }
    compuesto.push_back(actual);
    return compuesto;
}
```

El método Split se utilizo de tal manera que al momento que el usuario ingrese algún archivo este método separa los datos con un delimitador.

```
bool abrirArchivo(string nombreArchivo, ifstream &archivo)
{
    archivo.open(nombreArchivo);
    if (!archivo.is_open())
    {
        cout << "No se pudo abrir el archivo: " << nombreArchivo << endl;
        return false;
    }
    return true;
}
```

El método abrirArchivo: este método tiene como finalidad retornar un valor verdadero o falso para indicarle al sistema que si se logro abrir el archivo ingresado.

```
void procesarArchivo(string nombreArchivo, vector<Curso> &cursos, vector<Estudiante> &estudiantes, vector<Nota> &notas)
{
    ifstream archivo;
    if (!abrirArchivo(nombreArchivo, archivo))
    {
        return;
    }
    string linea;
    while (getline(archivo, linea))
    {
        vector<string> datos = split(linea, ',');
        if (nombreArchivo == "cursos.lfp" && datos.size() == 5)
        {
            int id = stoi(datos[0]);
            string nombre = datos[1];
            int creditos = stoi(datos[2]);
            int semestre = stoi(datos[3]);
            string carrera = datos[4];

            Curso c(id, nombre, creditos, semestre, carrera);
            cursos.push_back(c);
        }
        else if (nombreArchivo == "estudiantes.lfp" && datos.size() == 5)
        {
            int carnet = stoi(datos[0]);
            string nombre = datos[1];
            string apellido = datos[2];
            string carrera = datos[3];
            int semestre = stoi(datos[4]);

            Estudiante e(carnet, nombre, apellido, carrera, semestre);
            estudiantes.push_back(e);
        }
        else if (nombreArchivo == "notas.lfp" && datos.size() == 5)
```

El método procesar archivo nos permite recaudar los datos de los archivos ingresados al sistema, con este método se pueden crear los vectores de datos de estudiantes, cursos y notas que luego servirán para realizar los reportes.